Types of Errors

Introduction to computer programming

Management and Artificial Intelligence

Recap examples

Define an integer variable

1. Define an integer variable `a` and assign it the value 173
2. Print the value of `a`
3. Print the type of `a`

Define an integer variable

1. Define an integer variable `a` and assign it the value 173
2. Print the value of `a`
3. Print the type of `a`

```
a = 173

print("The value of a is", a)
print("The type of a is", type(a))
```

```
The value of a is 173
The type of a is <class 'int'>
```

Define a string variable

1. Define a string variable `s` and assign it the value "Hello, World!"
2. Print the value of `s`
3. Print the type of `s`

Define a string variable

1. Define a string variable `s` and assign it the value "Hello, World!"
2. Print the value of `s`
3. Print the type of `s`

```
s = "Hello, World!"

print("The value of s is", s)
print("The type of s is", type(s))
```

```
The value of s is Hello, World!
The type of s is <class 'str'>
```

Convert string to int

1. Define a string variable `s` with a value of "123".
2. Print value of the string `s`.
3. Print the type of the string `s`.
4. Convert the string `s` to an integer `d`.
5. Print the integer `d`.
6. Print the type of the integer `d`.

Convert string to int

1. Define a string variable `s` with a value of "123".
2. Print value of the string `s`.
3. Print the type of the string `s`.
4. Convert the string `s` to an integer `d`.
5. Print the integer `d`.
6. Print the type of the integer `d`.

```
s = "123"
print(s)
print(type(s))
d = int(s)
print(d)
print(type(d))
```

```
123
<class 'str'>
123
<class 'int'>
```

Types of Errors in Python

We can diferentiate errors in Python in three main categories:

1. Syntax Errors
2. Runtime Errors
3. Logical Errors

Syntax Errors

- They occur when Python cannot understand the code due to a violation of language rules.
- They happen **before** the program runs.
- If the syntax is wrong and the code is processed for execution, then Python will always detect it.
- They will be the most common type of error you will see!

If the syntax is not correct, Python cannot understand what the code is trying to do.

Syntax Errors

- They occur when Python cannot understand the code due to a violation of language rules.
- They happen **before** the program runs.
- If the syntax is wrong and the code is processed for execution, then Python will always detect it.
- They will be the most common type of error you will see!

If the syntax is not correct, Python cannot understand what the code is trying to do.

For instance, consider the following code:

```
print "Hello, World"
```

Can you spot the problem?

An example of syntax error

When attempting to run the code:

```
print "Hello, World"
```

```
  Cell In[5], line 1
    print "Hello, World"
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print
(...)?
```

An example of syntax error

When attempting to run the code:

```
print "Hello, World"
```

```
  Cell In[5], line 1
    print "Hello, World"
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print
(...)?
```

Python gives us a clear message about what went wrong:

1. The type of error: `SyntaxError`
2. The line number where the error occurred: line `1`
3. The error message: `Missing parentheses in call to 'print'`
4. A suggestion: `Did you mean print(...)?`

WARNING: Read the error message carefully! It can save you a lot of time!

What is a possible fix to the code?

```
print("Hello, World!")
```

As suggested by the Python error message, we need to add the **parantheses** to the function call to `print`.

Runtime Errors

- They occur when an error happens during execution, causing the program to stop.
- They happen **while** the program is running.
- The error may happen only for some data values! You may not see the error for some data values. **This makes them tricky!**
- They will be the second most common type of error we will see!

Python can run your code but it stops because it detects an error.

Runtime Errors

- They occur when an error happens during execution, causing the program to stop.
- They happen **while** the program is running.
- The error may happen only for some data values! You may not see the error for some data values. **This makes them tricky!**
- They will be the second most common type of error we will see!

Python can run your code but it stops because it detects an error.

For instance, consider the following code:

```
x = 10
y = 0
z = x / y
```

Can you spot the problem?

An example of runtime error

```
x = 10
y = 0
z = x / y
```

An example of runtime error

```
x = 10
y = 0
z = x / y
```

Python gives us a clear message about what went wrong:

1. The type of error: `ZeroDivisionError`
2. The line number where the error occurred: line `3`
3. The error message: `division by zero`

WARNING: Read the error message carefully! It can save you a lot of time!

What is a possible fix to the code?

Two options:

1. We use a different value for the denominator `y`.

```
x = 10
y = 2
z = x / y
print("x is", x, "\ny is", y, "\nz is", z)
```

2. We check if the denominator is zero before performing the division. *We will how to do this in the next lectures.*

Logical Errors

- The program runs but produces incorrect or unintended results.
- Happens **after** the program runs **without** errors.
- Since it does not raise an error, they are extremely **subtle** to find.
- To catch them, you need to reason about the **semantic** meaning of the code.

Logical Errors

- The program runs but produces incorrect or unintended results.
- Happens **after** the program runs **without** errors.
- Since it does not raise an error, they are extremely **subtle** to find.
- To catch them, you need to reason about the **semantic** meaning of the code.

For example, consider the following code:

```
a = 10
b = 2
sum_a_b = a - b
print("Sum of", a, "and", b, "is:", sum_a_b)
```

Can you spot the error?

Example of a logical error

```
a = 10
b = 2
sum_a_b = a - b
print("Sum of", a, "and", b, "is:", sum_a_b)
```

- Python is not able to detect the error. **Why?**
- The error is about the semantic of your code.
- Python does not what is the end goal of your code.
- You are only knowing what is the end goal, thus the expected result.

What is a possible fix to the code?

```
a = 10
b = 2
sum_a_b = a + b
print("Sum of", a, "and", b, "is:", sum_a_b)
```

We need the replace the operator `-` with `+` since our goal is to compute the sum of the two variables.

Additional examples

```
# syntax error
print(3+2
```

```
    Cell In[6], line 2
      print(3+2
                ^
  SyntaxError: incomplete input
```

FIX: close the parenthesis

```
print(3+2)
```

```
  5
```

```
# syntax error
print("hello world')
```

```
    Cell In[9], line 2
      print("hello world')
            ^
  SyntaxError: unterminated string literal (detected at line 2)
```

FIX: close the string with the same quote used to open it.

```
print("hello world")
```

```
# syntax error
44_cats = "cartoon"
```

```
    Cell In[10], line 2
      44_cats = "cartoon"
        ^
  SyntaxError: invalid decimal literal
```

FIX: do not use numbers as prefix of variable names

```
cats_44 = "cartoon"
```

```
# runtime error
print(my_var)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[11], line 2
      1 # runtime error
----> 2 print(my_var)

NameError: name 'my_var' is not defined
```

FIX: define the variable before using it!

```
my_variable = 42
print(my_variable)
```

```
42
```

```python
# runtime error
temperature = "25°C"
temperature += 10  # Attempting to add an integer to a temperature string
print(temperature)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[13], line 3
      1 # runtime error
      2 temperature = "25°C"
----> 3 temperature += 10  # Attempting to add an integer to a temperature
string
      4 print(temperature)

TypeError: can only concatenate str (not "int") to str
```

FIX: keep `temperature` as an integer

```python
temperature = 25
temperature += 10  # Attempting to add an integer to a temperature string
print(temperature, "°C")
```

```
35 °C
```

```python
# runtime error
number = 42
text = "Hello, World!"
result = number + text
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[15], line 4
      2 number = 42
      3 text = "Hello, World!"
----> 4 result = number + text

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

FIX: convert `number` to string before concatenate it to `text`

```python
number = 42
text = "Hello, World!"
result = str(number) + text
print(result)
```

```
42Hello, World!
```

```
# semantic error
radius = 10
area = 2 * 3.14159 * radius  # Incorrect formula for calculating the area of a circle
print(area)
```

```
62.8318
```

FIX: Use the proper formula $\pi * r^2$

```
radius = 10
area = 3.14159 * (radius ** 2)  # Correct formula for area of a circle
print(area)
```

```
314.159
```

```
# semantic error
price = 100
discount = 20  # 20% discount
final_price = price * (discount / 100)  # Incorrect application of % discount
print(final_price)
```

```
20.0
```

FIX: Use the correct formula!

```
price = 100
discount = 20  # 20% discount
final_price = price - (price * discount / 100) # Correct application of % discount
print(final_price)
```

```
80.0
```